

Cargador de Videojuegos a Qdrant - Documentación Técnica

Índice

1. [Descripción General](#)
 2. [Arquitectura del Flujo](#)
 3. [Requisitos Previos](#)
 4. [Configuración de Nodos](#)
 5. [Flujo de Datos](#)
 6. [Ejecución del Workflow](#)
-

Descripción General

Este workflow de n8n implementa un sistema completo de carga de datos de videojuegos a una base de datos vectorial Qdrant. El flujo procesa información de 200 videojuegos, filtra los juegos de género RPG, transforma los datos en embeddings vectoriales utilizando Ollama, y los almacena en Qdrant para búsquedas semánticas posteriores.

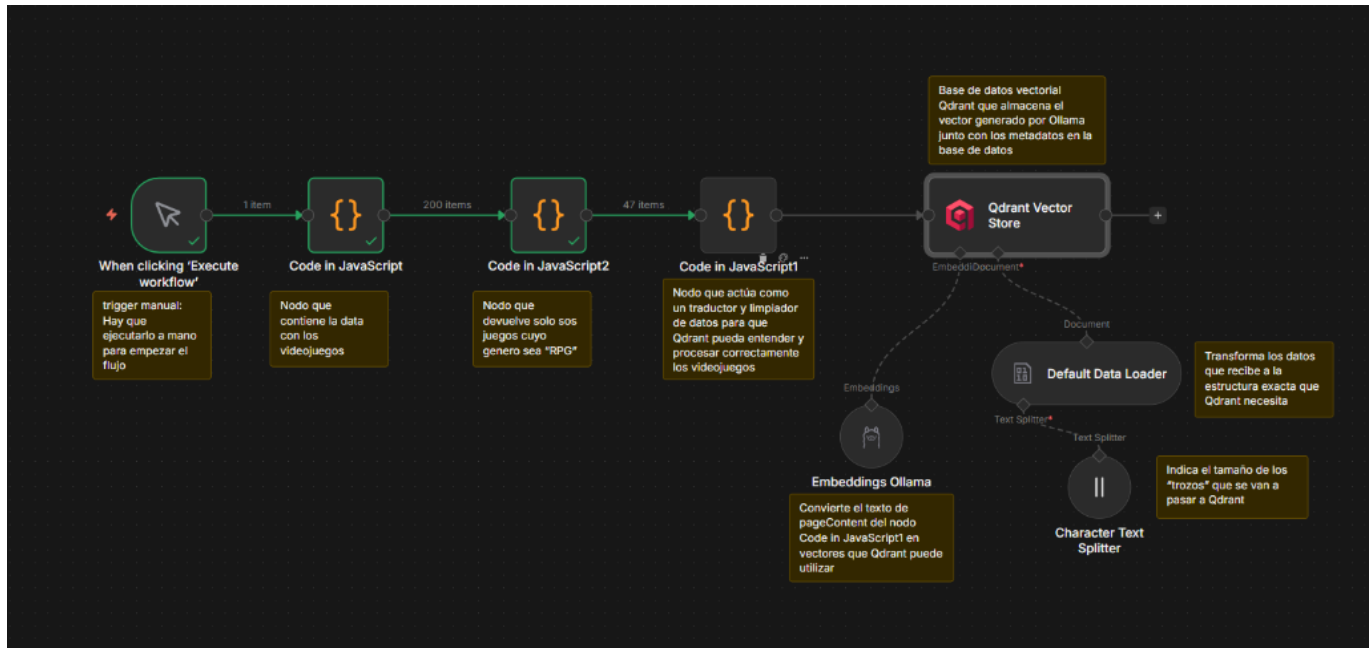
Objetivo Principal

Crear una base de datos vectorial que permita realizar búsquedas semánticas inteligentes sobre un catálogo de videojuegos, facilitando recomendaciones basadas en características como género, temática y plataforma.

Tecnologías Utilizadas

- **n8n**: Plataforma de automatización de workflows
 - **Qdrant**: Base de datos vectorial para búsquedas semánticas
 - **Ollama**: Modelo de embeddings local (nomic-embed-text)
 - **JavaScript**: Lenguaje de transformación de datos
-

Arquitectura del Flujo



El workflow está compuesto por **7 nodos principales** y **7 nodos de documentación** (Sticky Notes) que se ejecutan en el siguiente orden:

```

When clicking 'Execute workflow'
↓
Code in JavaScript (Datos de videojuegos)
↓
Code in JavaScript2 (Filtro RPG)
↓
Code in JavaScript1 (Transformación para embeddings)
↓
Qdrant Vector Store ← Embeddings Ollama
↑
Default Data Loader ← Character Text Splitter
  
```

⚙️ Requisitos Previos

1. Credenciales Necesarias

Qdrant API

- **ID de Credencial:** `QRPb7Z2dQ4rSVUeh`
- **Nombre:** QdrantApi account
- **Configuración requerida:**
 - URL del servidor Qdrant
 - API Key de autenticación

Ollama API

- **ID de Credencial:** `XFDHqbOdPU9cx52k`
- **Nombre:** Ollama account

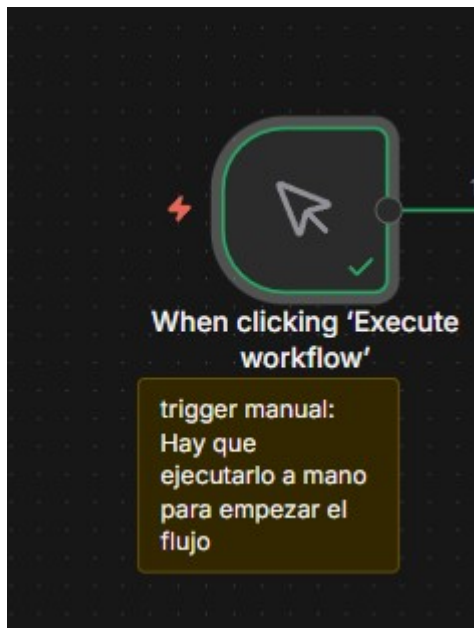
- **Configuración requerida:**
 - URL del servidor Ollama local
 - Modelo instalado: `nomic-embed-text:latest`

2. Colección en Qdrant

- **Nombre de la colección:** `videojuegos`
 - **Dimensión del vector:** Debe coincidir con el modelo `nomic-embed-text` (768 dimensiones)
-

Configuración de Nodos

1 When clicking 'Execute workflow'



Tipo: Manual Trigger

Versión: 1

Descripción: Nodo trigger manual que inicia la ejecución del flujo.

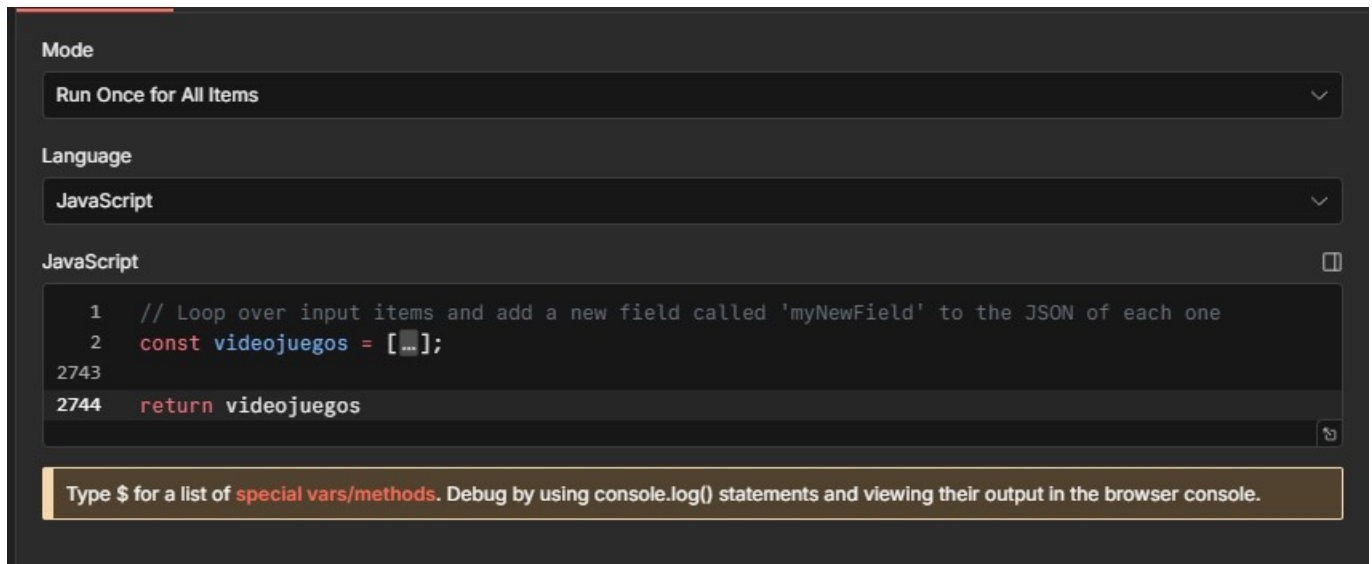
Configuración:

- No requiere parámetros adicionales
- Se ejecuta manualmente desde la interfaz de n8n

Función:

- Punto de entrada del workflow
 - Permite ejecutar el flujo bajo demanda
 - Ideal para cargas iniciales o actualizaciones controladas
-

2 Code in JavaScript



Tipo: Code Node (JavaScript)

Versión: 2

Items procesados: 1 → 200

Descripción: Contiene el dataset completo de 200 videojuegos con toda su información estructurada.

Configuración:

```
// Mode: Run Once for All Items
// Language: JavaScript

const videojuegos = [
  {
    id: "1",
    nombre: "The Witcher 3: Wild Hunt",
    genero: "RPG",
    plataforma: "PC/PS5/Xbox",
    descripcion: "Aventura épica de mundo abierto...",
    precio: "39.99",
    tematica: "Fantasía Medieval",
    desarrollador: "CD Projekt Red",
    año: "2015",
    url_imagen: "https://cdn.cloudflare.steamstatic.com/..."
  },
  // ... 199 videojuegos más
];

return videojuegos;
```

Estructura de Datos: Cada videojuego contiene los siguientes campos:

- **id:** Identificador único
- **nombre:** Título del juego
- **genero:** Categoría principal
- **plataforma:** Plataformas disponibles

- **descripcion**: Descripción detallada
- **precio**: Precio en USD
- **tematica**: Temática del juego
- **desarrollador**: Estudio desarrollador
- **año**: Año de lanzamiento
- **url_imagen**: URL de la imagen de portada

Salida: Array de 200 objetos con información de videojuegos

3 Code in JavaScript2

The screenshot shows a code editor with the following content:

```

Run Once for All Items

Language
JavaScript

JavaScript

2  try {
3    const items = $input.all();
4
5    const data = items.filter(item=>{return item.json.genero.includes("RPG")});
6
7    return data;
8  } catch (error) {
9    console.log("Error", error);
10   return [{
11     id:"",
12     nombre:"",
13     genero:"",
14     plataforma:"",
15     descripcion:"",
16     precio:"",
17     tematica:"",
18     desarrollador:"",
19     año:"",
20     url_imagen:""
21   }]

```

At the bottom, a message reads: "Type \$ for a list of special vars/methods. Debug by using console.log() statements and viewing their output in the browser console."

Tipo: Code Node (JavaScript)

Versión: 2

Items procesados: 200 → 47

Descripción: Filtra únicamente los videojuegos cuyo género incluye "RPG".

Configuración:

```

// Mode: Run Once for All Items
// Language: JavaScript

try {
  const items = $input.all();

```

```
const data = items.filter(item => {
  return item.json.genero.includes("RPG")
});

return data;
} catch (error) {
  console.log("Error", error);
  return [{
    id: "",
    nombre: "",
    genero: "",
    plataforma: "",
    descripcion: "",
    precio: "",
    tematica: "",
    desarrollador: "",
    año: "",
    url_imagen: ""
  }]
}
```

Lógica de Filtrado:

- Recibe todos los items del nodo anterior
- Aplica filtro: `genero.includes("RPG")`
- Manejo de errores con try-catch
- Retorna objeto vacío en caso de error

Géneros Filtrados:

- RPG
- JRPG
- Action RPG
- RPG/Acción
- RPG por turnos
- Estrategia/RPG

Salida: 47 videojuegos de género RPG

4 Code in JavaScript1

Edit JavaScript

```

9
10 // Usamos map para transformar cada item individualmente
11 return items.map(item => {
12   const data = item.json;
13
14   const textoParaEmbeddings = `juego:${data.nombre}.genero:${data.genero}.tematica:${data.tematica}.plataforma:${data.plataforma}`
15   .replace(/\n\t\r/g, '')
16   .replace(/\s+/g, '')
17   .trim();
18
19   return {
20     pagecontent: textoParaEmbeddings,
21     metadata: {
22       id: data.id,
23       nombre: data.nombre,
24       genero: data.genero,
25       plataforma: data.plataforma,
26       precio: data.precio,
27       imagen: data.url_imagen
28     }
29   };
30 });
31 } catch (error) {
32   console.log("Error", error);
33   return {
34     pagecontent: "",
35     metadata: {
36       id: "",
37       nombre: "",
38       genero: "",
39       plataforma: "",
40       precio: 0,
41       imagen: ""
42     }
43   };
44 }
45

```

Tipo: Code Node (JavaScript)**Versión:** 2**Items procesados:** 47 → 47**Descripción:** Transforma los datos al formato requerido por Qdrant, creando el texto para embeddings y estructurando los metadatos.**Configuración:**

```

try {
  const items = $input.all()

  // Usamos map para transformar cada item individualmente
  return items.map(item => {
    const data = item.json;

    const textoParaEmbeddings =
`juego:${data.nombre}.genero:${data.genero}.tematica:${data.tematica}.plataforma:${data.plataforma}`
    .replace(/\n\t\r/g, '')
    .replace(/\s+/g, '')
    .trim();

    return {
      pagecontent: textoParaEmbeddings,

```

```

        metadata: {
            id: data.id,
            nombre: data.nombre,
            genero: data.genero,
            plataforma: data.plataforma,
            precio: data.precio,
            imagen: data.url_imagen
        }
    });
} catch (error) {
    console.log("Error", error);
    return {
        pagecontent: "",
        metadata: {
            id: "",
            nombre: "",
            genero: "",
            plataforma: "",
            precio: 0,
            imagen: ""
        }
    }
}
}

```

Proceso de Transformación:

1. Creación del texto para embeddings (pagecontent):

- Formato: `juego:nombre.genero:tipo.tematica:tema.plataforma:plat`
- Eliminación de saltos de línea (`\n`, `\t`, `\r`)
- Eliminación de espacios múltiples
- Trim de espacios al inicio y final

2. Estructura de metadatos:

- `id`: Identificador único
- `nombre`: Nombre del juego
- `genero`: Género del juego
- `plataforma`: Plataformas disponibles
- `precio`: Precio del juego
- `imagen`: URL de la imagen

Ejemplo de Salida:

```

{
  "pagecontent":
  "juego:TheWitcher3:WildHunt.genero:RPG.tematica:FantasíaMedieval.plataforma:PC/PS5
/Xbox",
  "metadata": {
    "id": "1",

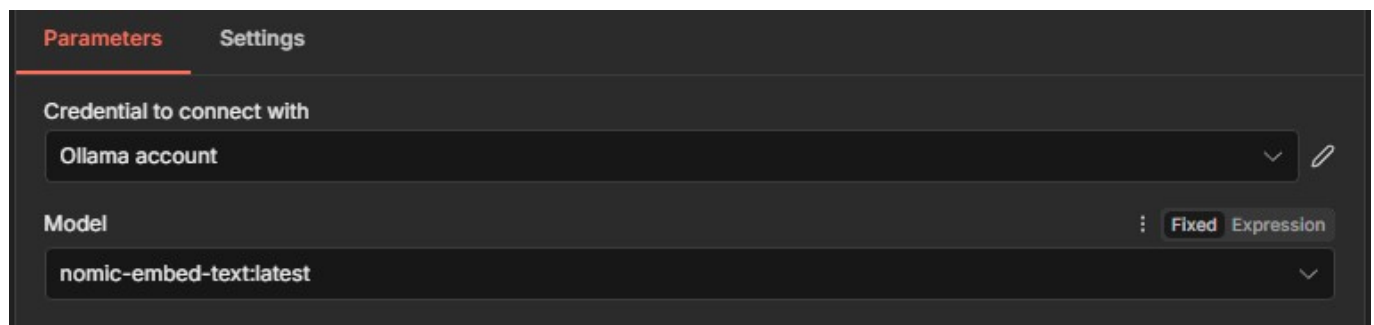
```



```
"nombre": "The Witcher 3: Wild Hunt",
"genero": "RPG",
"plataforma": "PC/PS5/Xbox",
"precio": "39.99",
"imagen": "https://cdn.cloudflare.steamstatic.com/..."
}
```

Salida: 47 objetos transformados con `pagecontent` y `metadata`

5 Embeddings Ollama



Tipo: Embeddings Ollama

Versión: 1

Descripción: Convierte el texto de `pageContent` en vectores numéricos que Qdrant puede utilizar para búsquedas semánticas.

Configuración:

- **Credential:** Ollama account (XFDHqbOdPU9cx52k)
- **Model:** `nomic-embed-text:latest`

Características del Modelo:

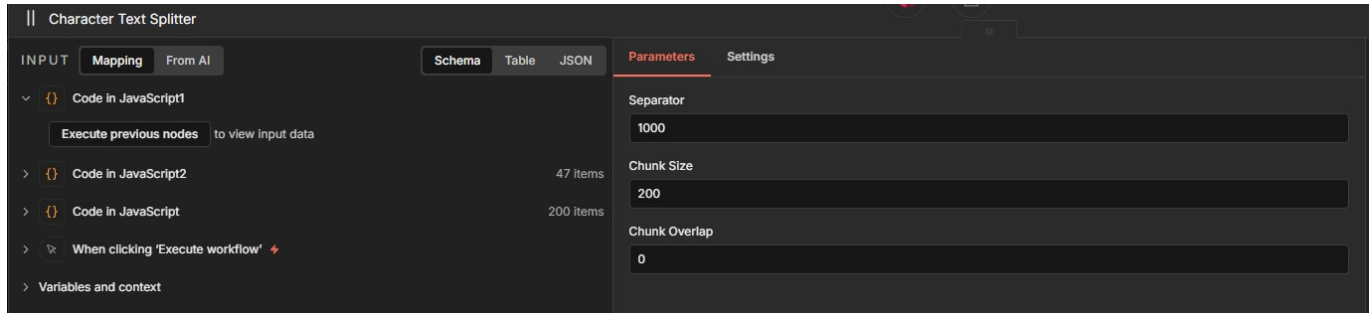
- **Dimensiones:** 768
- **Tipo:** Modelo de embeddings de texto
- **Ventajas:**
 - Ejecución local (sin costos de API)
 - Rápido y eficiente
 - Optimizado para búsquedas semánticas
 - Soporte multilingüe

Proceso:

1. Recibe el texto de `pagecontent` de cada item
2. Genera un vector de 768 dimensiones
3. El vector captura el significado semántico del texto
4. Se conecta directamente con Qdrant Vector Store

Conexión: `ai_embedding` → Qdrant Vector Store

6 Character Text Splitter



Tipo: Text Splitter

Versión: 1

Descripción: Define el tamaño de los "chunks" o fragmentos de texto que se procesarán.

Configuración:

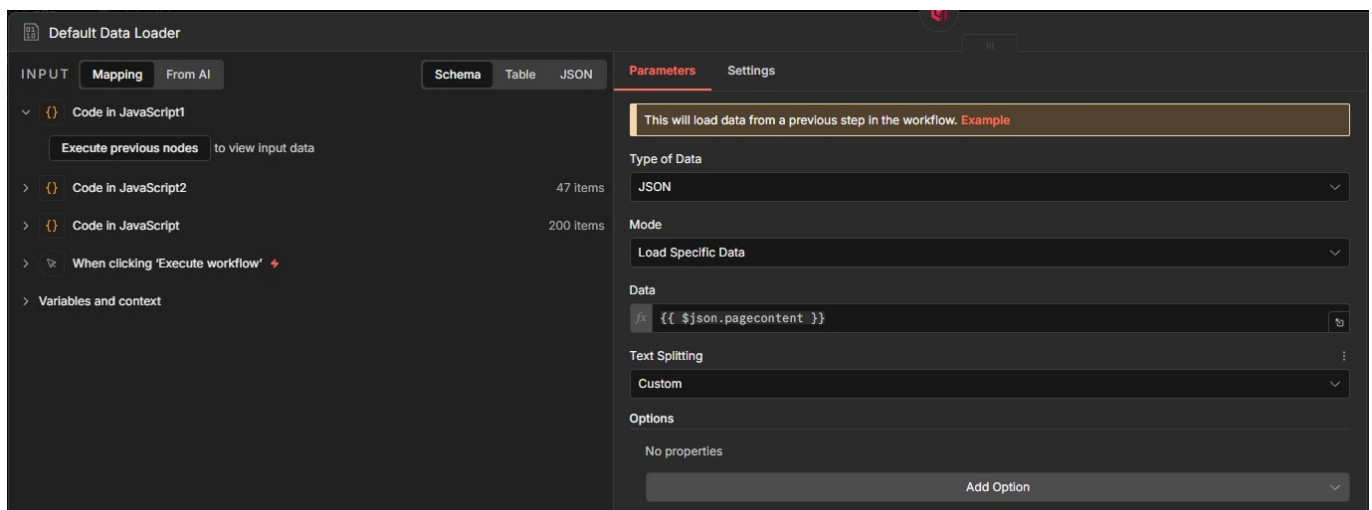
- **Separator:** 1000
- **Chunk Size:** 200 caracteres
- **Chunk Overlap:** 0 (por defecto)

Función:

- Divide textos largos en fragmentos manejables
- En este caso, como los textos son cortos (nombres de juegos + metadata), cada ítem se procesa completo
- Asegura que no se excedan los límites del modelo de embeddings

Conexión: ai_textSplitter → Default Data Loader

7 Default Data Loader



Tipo: Document Default Data Loader

Versión: 1.1

Descripción: Transforma los datos al formato exacto que Qdrant necesita para almacenar documentos.

Configuración:

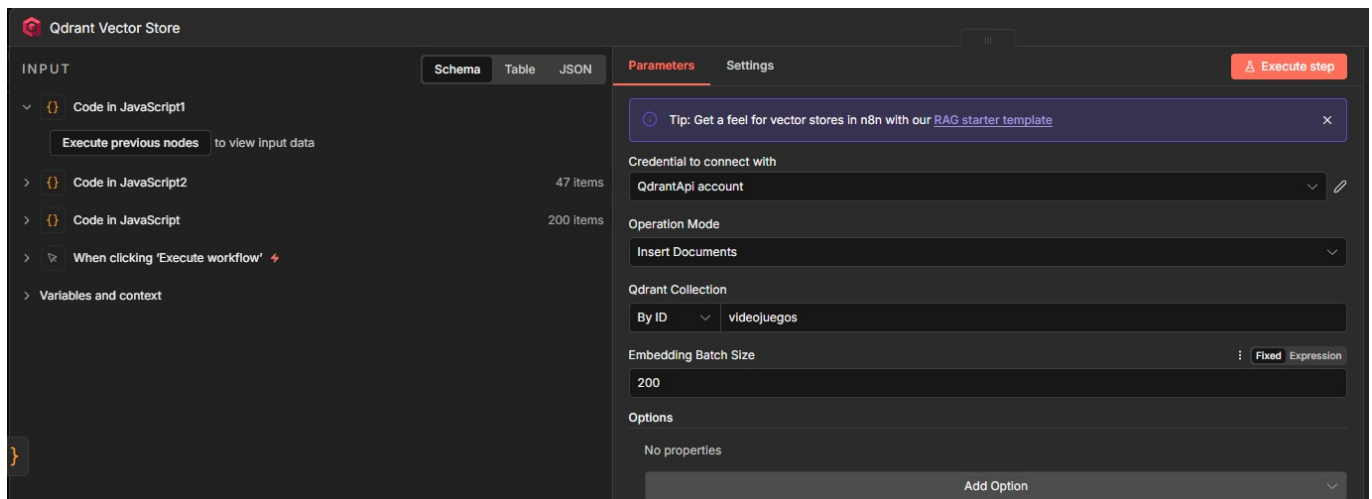
- **JSON Mode:** Expression Data
- **JSON Data:** `={{ $json.pagecontent }}`
- **Text Splitting Mode:** Custom
- **Options:** No properties

Proceso:

1. Extrae el campo `pagecontent` de cada item
2. Lo convierte en un documento procesable
3. Aplica el text splitter configurado
4. Prepara el formato para Qdrant

Conexión: `ai_document` → Qdrant Vector Store

8 Qdrant Vector Store



Tipo: Qdrant Vector Store

Versión: 1.3

Items procesados: 47

Descripción: Base de datos vectorial que almacena los embeddings generados junto con los metadatos de cada videojuego.

Configuración:

- **Credential:** QdrantApi account (`QRPb7Z2dQ4rSVUeh`)
- **Operation Mode:** Insert Documents
- **Qdrant Collection:** `videojuegos` (By ID)
- **Embedding Batch Size:** `200`
- **Options:** No properties

Conexiones de Entrada:

1. **Main Input:** Datos del nodo Code in JavaScript1
2. **ai_embedding:** Embeddings Ollama
3. **ai_document:** Default Data Loader

Proceso de Inserción:

1. Recibe los 47 items transformados
2. Para cada item:
 - Obtiene el vector de embeddings (768 dimensiones)
 - Extrae los metadatos
 - Crea un punto en Qdrant con:
 - **id:** Identificador único
 - **vector:** Array de 768 números
 - **payload:** Metadatos del juego
3. Inserta todos los puntos en la colección **videojuegos**

Estructura en Qdrant:

```
{
  "id": "1",
  "vector": [0.123, -0.456, 0.789, ...], // 768 dimensiones
  "payload": {
    "id": "1",
    "nombre": "The Witcher 3: Wild Hunt",
    "genero": "RPG",
    "plataforma": "PC/PS5/Xbox",
    "precio": "39.99",
    "imagen": "https://cdn.cloudflare.steamstatic.com/..."
  }
}
```

Resultado: 47 videojuegos RPG almacenados en Qdrant con búsqueda semántica habilitada

Flujo de Datos

Diagrama de Transformación

1. TRIGGER MANUAL

Entrada: Ninguna

Salida: Señal de inicio

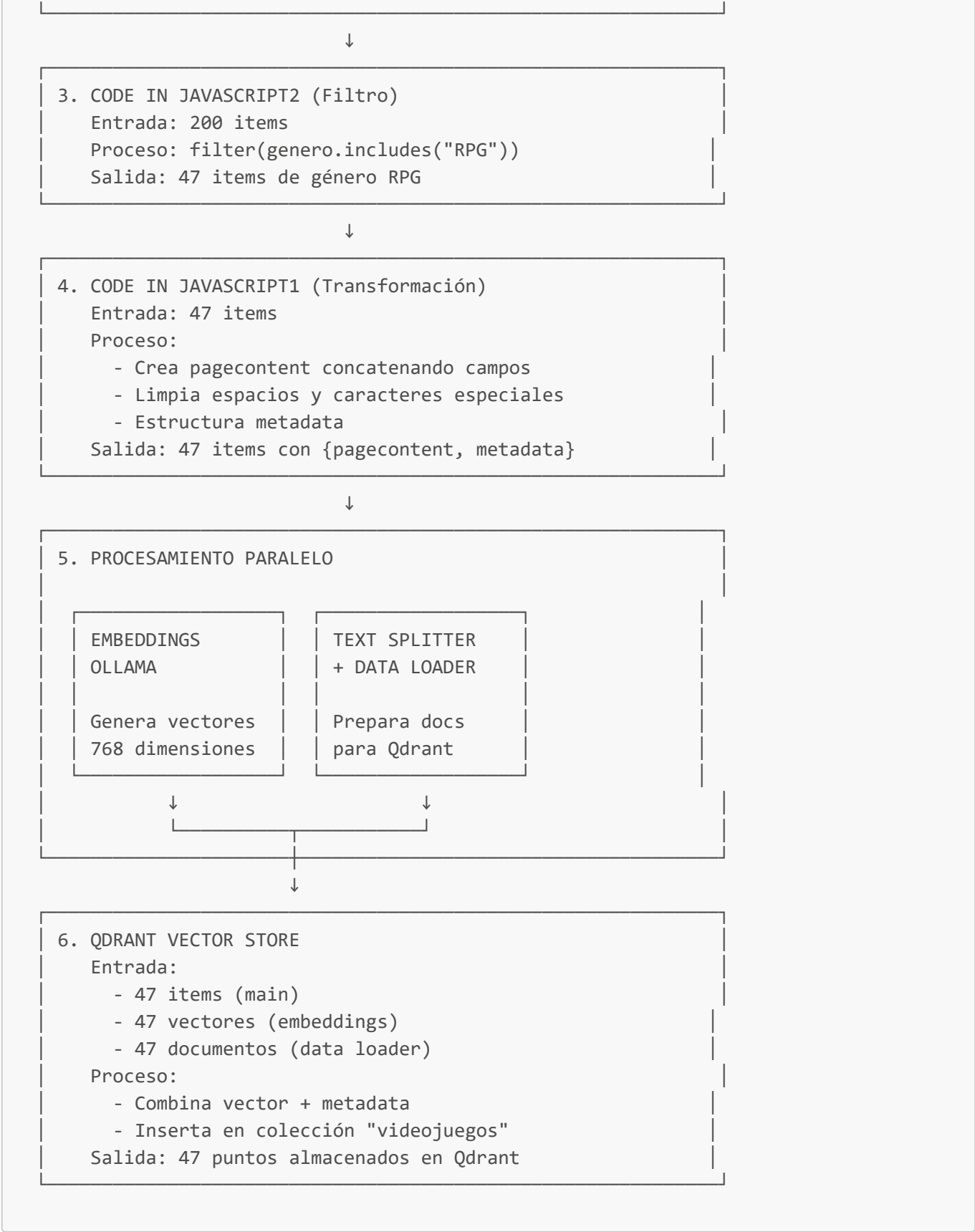


2. CODE IN JAVASCRIPT (Dataset)

Entrada: Ninguna

Proceso: Retorna array de 200 videojuegos

Salida: 200 items con estructura completa



Resumen de Transformaciones

Nodo	Items Entrada	Items Salida	Transformación
Trigger	0	1	Inicio del flujo
Code JS	1	200	Genera dataset

Nodo	Items Entrada	Items Salida	Transformación
Code JS2	200	47	Filtra RPG
Code JS1	47	47	Transforma formato
Embeddings	47	47	Genera vectores
Text Splitter	47	47	Divide texto
Data Loader	47	47	Prepara docs
Qdrant	47	47	Almacena en BD

▶ Ejecución del Workflow

Pasos para Ejecutar

1. Verificar Requisitos:

```
# Verificar que Ollama esté corriendo
curl http://localhost:11434/api/tags

# Verificar que el modelo esté instalado
ollama list | grep nomic-embed-text
```

2. Verificar Qdrant:

```
# Verificar conexión a Qdrant
curl http://localhost:6333/collections

# Verificar que la colección existe
curl http://localhost:6333/collections/videojuegos
```

3. Ejecutar en n8n:

- Abrir el workflow en n8n
- Click en el nodo "When clicking 'Execute workflow'"
- Click en "Execute Node" o "Execute Workflow"
- Observar la ejecución en tiempo real

4. Monitorear Ejecución:

- Cada nodo mostrará el número de items procesados
- Los nodos exitosos se marcarán en verde
- Los errores se mostrarán en rojo con detalles

Tiempo de Ejecución Estimado

- **Trigger:** Instantáneo
- **Code in JavaScript:** < 1 segundo
- **Code in JavaScript2:** < 1 segundo
- **Code in JavaScript1:** < 1 segundo
- **Embeddings Ollama:** 5-10 segundos (47 items)
- **Qdrant Insert:** 2-5 segundos
- **Total:** ~10-20 segundos

Verificación de Resultados

```
# Verificar cantidad de puntos en Qdrant
curl http://localhost:6333/collections/videojuegos

# Buscar un videojuego específico
curl -X POST http://localhost:6333/collections/videojuegos/points/scroll \
-H 'Content-Type: application/json' \
-d '{
  "limit": 10,
  "with_payload": true,
  "with_vector": false
}'
```

Estadísticas del Flujo

Datos Procesados

- **Total de videojuegos en dataset:** 200
- **Videojuegos filtrados (RPG):** 47
- **Dimensiones del vector:** 768
- **Tamaño de chunk:** 200 caracteres
- **Batch size de embeddings:** 200

Géneros RPG Incluidos

- RPG (Role-Playing Game)
- JRPG (Japanese RPG)
- Action RPG
- RPG/Acción
- RPG por turnos
- Estrategia/RPG
- MMORPG

Distribución de Juegos RPG

- **Fantasía Medieval:** ~15 juegos
- **Ciencia Ficción:** ~8 juegos
- **Fantasía Oscura:** ~6 juegos

- **Contemporáneo:** ~5 juegos
 - **Mitología:** ~4 juegos
 - **Otros:** ~9 juegos
-

Casos de Uso

Una vez cargados los datos en Qdrant, se pueden realizar:

1. Búsquedas Semánticas

```
// Buscar juegos similares a "fantasía medieval con mundo abierto"
query_vector = embeddings("fantasía medieval mundo abierto")
results = qdrant.search(query_vector, limit=5)
```

2. Recomendaciones

```
// Encontrar juegos similares a The Witcher 3
game_vector = qdrant.get_vector("1") // ID de The Witcher 3
similar = qdrant.search(game_vector, limit=10)
```

3. Filtros Combinados

```
// Buscar RPG de PS5 baratos
qdrant.search(
  query_vector,
  filter: {
    plataforma: "PS5",
    precio: { $lt: 40 }
  }
)
```

Mantenimiento y Actualizaciones

Agregar Nuevos Videojuegos

1. Editar el nodo "Code in JavaScript"
2. Agregar nuevos objetos al array `videojuegos`
3. Ejecutar el workflow completo
4. Qdrant insertará solo los nuevos registros

Actualizar Videojuegos Existentes

1. Modificar los datos en "Code in JavaScript"

2. Cambiar el modo de Qdrant a "Update"
3. Ejecutar el workflow

Limpiar la Colección

```
# Eliminar todos los puntos
curl -X POST http://localhost:6333/collections/videojuegos/points/delete \
-H 'Content-Type: application/json' \
-d '{
  "filter": {
    "must": [
      {
        "key": "genero",
        "match": {
          "value": "RPG"
        }
      }
    ]
  }
}'
```

Notas Técnicas

Optimizaciones Aplicadas

1. **Limpieza de Texto:** Eliminación de espacios y caracteres especiales para mejorar la calidad de los embeddings
2. **Batch Processing:** Procesamiento en lotes para optimizar el rendimiento
3. **Manejo de Errores:** Try-catch en todos los nodos de código
4. **Metadatos Selectivos:** Solo se almacenan los campos relevantes para búsquedas

Limitaciones Conocidas

1. **Modelo Local:** Requiere Ollama corriendo localmente
2. **Filtro Estático:** El filtro de RPG está hardcodeado
3. **Sin Deduplicación:** No verifica duplicados antes de insertar
4. **Idioma:** Optimizado para español/inglés

Mejoras Futuras

- ☐ Parametrizar el filtro de género
- ☐ Agregar validación de datos
- ☐ Implementar deduplicación
- ☐ Agregar logging detallado
- ☐ Crear endpoint de consulta
- ☐ Implementar actualización incremental

Referencias

- [Documentación de n8n](#)
- [Qdrant Documentation](#)
- [Ollama Models](#)
- [Nomic Embed Text](#)

 Autor: Guillermo Bazan Diaz

Proyecto: Cargador de Videojuegos Qdrant

Fecha: Enero 2026

Versión: 1.0.0

Licencia

Este proyecto es parte de un ejercicio académico para la asignatura de Agentes IA 2025-2026.